

LSNB Réseau - Guide de reprise

Pour l'équipe de développement, y compris les personnes qui débutent. État du code au 5 septembre 2026, commit de référence f343098.

1. Comprendre l'existant

- **Code GitHub** : github.com/Mounir1200/lsnb-alumni

- **Site sur Render** : lsnb-alumni-web.onrender.com

Le projet est un **prototype de réseau pour les élèves et anciens élèves du Lycée Scientifique National de Bobo-Dioulasso**. Il permet de découvrir des parcours, de créer un profil et de préparer des échanges de mentorat. Il reste à compléter avant une ouverture générale.

Ce qui est déjà développé

Fonction	État dans le code
Accueil et navigation	Pages principales, affichage adapté au mobile et animations. L'accueil présente encore des profils fictifs.
Annuaire et fiches	Recherche par texte, filtres domaine/pays/mentorat et fiches individuelles. Lecture des profils Supabase si disponibles et autorisés.
Comptes	Inscription, confirmation par e-mail, connexion, renvoi de confirmation et déconnexion avec Supabase.
Espace membre	Consultation et modification de son profil, disponibilité pour le mentorat et choix de visibilité du contact.
Photos	Ajout, remplacement et suppression ; JPEG, PNG ou WebP, 4 Mo maximum.
Mise en relation	Formulaire relié à une table de demandes. Le parcours réel reste à tester ; la boîte de réception et les réponses restent à développer.
Serveur et hébergement	Socle d'API Node.js et configuration de deux services Render. L'API ne gère pas encore les fonctions métier.

Bien distinguer démonstration et données réelles

Les **huit profils d'exemple sont fictifs**. L'accueil en affiche toujours une sélection, même avec Supabase configuré. L'annuaire utilise aussi ces exemples si Supabase est absent, si aucun profil n'est renvoyé ou si une erreur survient. Voir des exemples ne suffit donc pas à conclure que la base fonctionne.

Sans Supabase, l'inscription conserve seulement un aperçu de profil dans le navigateur : **aucun compte distant n'est créé**, aucun mot de passe n'est enregistré et ce profil n'est pas ajouté à l'annuaire. Une demande adressée à un profil fictif n'envoie aucun message réel.

Organisation : Supabase et Render sont déjà configurés et restent administrés par le responsable du projet. L'équipe développe le code et lui transmet les besoins liés à ces services. Les comportements décrits ci-dessous sont fondés sur le dépôt et les vérifications locales ; les parcours avec de vrais comptes restent à valider.

2. Reprendre le projet en local

Les composants, simplement

- **Frontend** : l'interface affichée dans le navigateur, écrite avec React et TypeScript. Vite lance le site en local et prépare les fichiers à publier ; Tailwind CSS gère les styles.
- **Supabase** : le service qui gère les comptes, la base de données et les photos.
- **Backend / API** : un serveur Node.js avec Fastify. Une API est un ensemble d'adresses auxquelles un programme peut envoyer des requêtes.

Aujourd'hui, l'interface communique directement avec Supabase. L'API séparée ne fournit que /health (état du serveur) et /api/v1 (identification). Le passage des fonctions métier par cette API est une évolution à construire.

Où modifier quoi ?

Dossier ou fichier	Utilité
src/pages/ et src/App.tsx	Pages et adresses du site : /annuaire, /rejoindre, /connexion, /espace et /espace/modifier, notamment.
src/components/ et src/styles.css	Éléments visuels réutilisables et styles.
src/lib/ et src/auth/	Accès Supabase, profils, photos et session du membre connecté.
src/data/alumni.ts	Profils fictifs de démonstration.
api/src/	Code du serveur et son test de disponibilité.
supabase/migrations/	Fichier SQL qui crée les tables et les règles d'accès.
render.yaml	Configuration de déploiement des deux services Render.

Démarrage rapide

Installer **Git et Node.js 24 avec npm**. Node exécute les outils JavaScript ; npm installe leurs dépendances. Render est configuré avec Node 24.14.1.

Dans un terminal :

```
git clone https://github.com/Mounir1200/lsnb-alumni.git
cd lsnb-alumni
npm ci
npm run dev
```

Si le dépôt est déjà sur votre ordinateur, ouvrez simplement un terminal dans son dossier. `npm ci` installe les versions prévues par le projet. Ouvrez ensuite l'adresse affichée par Vite, généralement `http://localhost:5173`. Sans configuration Supabase, le site démarre en démonstration.

API facultative pour les fonctions actuelles du site. Pour la lancer, ouvrir un deuxième terminal à la racine du projet :

```
npm --prefix api ci
npm run dev:api
```

Consulter `http://localhost:4000/health` pour vérifier que ce serveur répond. Le laisser ouvert pendant son utilisation.

3. Comprendre le stockage et les accès

Où sont enregistrées les données ?

Render héberge le site et l'API. Supabase conserve les comptes, les données des membres et les photos.

Le navigateur envoie actuellement les données directement à Supabase. La base PostgreSQL organise les informations dans des tables, comparables à des feuilles de calcul reliées entre elles.

Emplacement	Contenu
Supabase Auth (auth.users)	Compte de connexion, e-mail et identifiant unique du membre. L'authentification et les mots de passe sont gérés par Supabase Auth.
Table profiles	Nom, promotion, parcours, ville/pays, mentorat et adresse de la photo (photo_url).
Table profile_contacts	E-mail de contact et choix de visibilité, séparés du profil.
Table connection_requests	Expéditeur, destinataire, message et statut de la demande : en attente, acceptée ou refusée.
Supabase Storage : avatars	Fichiers des photos, dans un espace de stockage appelé « bucket ».

L'identifiant du compte relie ces informations au bon membre. À l'inscription, une action automatique en base crée le profil et ses coordonnées. Les modifications enregistrées dans Supabase persistent après fermeture du navigateur. Les exemples fictifs restent dans `src/data/alumni.ts` ; l'aperçu d'inscription en mode démo reste seulement dans le `localStorage` du navigateur.

Comment la photo est-elle stockée ?

1. Le site vérifie le fichier : **JPEG, PNG ou WebP, 4 Mo maximum**. Une fois le membre connecté, il l'envoie dans `avatars`, au chemin `<identifiant-du-membre>/avatar`.
2. Supabase fournit l'adresse publique du fichier. **La table profiles conserve uniquement cette adresse dans photo_url**, puis le site la lit pour afficher l'image.
3. Remplacer la photo écrase le fichier au même emplacement et actualise son adresse d'affichage. Supprimer la photo retire le fichier et vide `photo_url`.

Avant confirmation de l'e-mail : si aucune session n'est encore ouverte, la photo attend dans **IndexedDB**, un stockage local du navigateur, associée à l'e-mail d'inscription. Elle peut être reprise pendant sept jours, dans le même navigateur. Le code prévoit de supprimer cette copie après l'envoi, ou lors de sa lecture si elle a expiré. Sur un autre appareil, il faut sélectionner la photo à nouveau.

Repères dans le code : `src/lib/avatarRepository.ts` pour l'envoi/suppression ;
`src/lib/pendingAvatarStore.ts` pour l'attente locale.

Qui peut accéder à quoi ?

Les règles **RLS** sont les permissions appliquées par la base : profils actifs lisibles par les membres connectés ; modification de son propre profil ; e-mail accessible à son propriétaire ou aux membres connectés si sa visibilité est activée ; demandes lisibles par leurs deux participants. **Les photos sont publiques pour toute personne ayant leur URL**, même sans connexion. L'envoi, le remplacement et la suppression sont réservés au propriétaire.

4. Ce qu'il reste à développer

Nouvelles fonctionnalités prévues

Ces trois ajouts font partie des évolutions demandées ; **ils ne sont pas encore développés**.

- **Connexion avec Google.** Ajouter « Continuer avec Google » à l'inscription et à la connexion, via Supabase. Après la première connexion, demander les informations LSNB manquantes. Vérifier aussi le cas d'une personne qui possède déjà un compte avec la même adresse e-mail.
- **Liens sur le profil.** Permettre d'ajouter, modifier et supprimer un lien LinkedIn et un lien de portfolio. Enregistrer ces champs dans la base et les afficher sur la fiche du membre. Vérifier que les adresses sont valides et que les champs restent facultatifs.
- **Highlight de la semaine.** Mettre en avant **deux alumni, un homme et une femme, pendant une semaine**. Générer par IA une section qui présente leurs parcours à partir des informations de leurs profils, sans inventer de faits. Enregistrer la sélection, les dates et le texte en base pour garder le même contenu toute la semaine ; renouveler ensuite la sélection. La génération doit se faire côté serveur, une fois par semaine, et non à chaque visite.

Détails du highlight à définir avec l'équipe : sélection manuelle ou automatique, jour de renouvellement, information de genre déclarée volontairement par le membre (aucun champ prévu aujourd'hui ; ne pas la déduire du prénom ou de la photo), accord des personnes mises en avant et relecture du texte IA. Prévoir aussi le cas où deux profils éligibles ne sont pas disponibles ou si la génération échoue.

Finaliser aussi les parcours existants

1. **Fiabiliser les données affichées.** Relier l'accueil et ses filtres aux vrais profils. Dans l'annuaire, distinguer une base vide, une erreur technique et le mode démonstration.
2. **Terminer la mise en relation.** Tester l'enregistrement d'une demande avec deux comptes, puis ajouter les demandes reçues/envoyées, l'acceptation, le refus et le suivi. Prévoir les notifications si elles sont retenues. L'interface n'affiche pas encore les coordonnées autorisées des autres membres.
3. **Compléter la gestion du compte.** Ajouter le mot de passe oublié, son changement et la suppression du compte. Le renvoi de confirmation d'e-mail existant ne réinitialise pas le mot de passe.
4. **Préparer l'ouverture aux membres.** Définir puis développer la vérification des inscriptions et la modération. Clarifier avec l'équipe la visibilité souhaitée des photos et des coordonnées, puis aligner l'interface et la page de confidentialité.
5. **Renforcer la validation.** Ajouter des tests des parcours principaux et des droits d'accès. Vérifier le mobile, la navigation au clavier et les erreurs. Le seul test automatisé actuel contrôle /health de l'API.

Les fonctions serveur nécessaires, notamment la génération IA et son exécution hebdomadaire, restent à construire dans l'API. Conserver les clés des services IA uniquement côté serveur.

Répartition du travail : l'équipe développe les écrans, la logique et les migrations proposées. Le responsable configure la connexion Google côté Supabase, applique les ajouts de champs/tables et gère les clés et réglages de déploiement nécessaires au highlight.

5. Valider et travailler en équipe

Accès de développement et administration

Le responsable du projet garde la main sur Supabase et Render : réglages, clés, droits d'accès, changements de base et mise en production. Pour les essais connectés, lui demander l'environnement autorisé, son URL et sa clé publique. Copier `.env.example` en `.env.local`, renseigner `VITE_SUPABASE_URL` et `VITE_SUPABASE_ANON_KEY`, puis relancer le site. `VITE_API_URL` est prévue pour plus tard, mais n'est pas utilisée aujourd'hui.

Les fichiers `.env` restent locaux. **Aucune clé secrète dans GitHub ou dans une variable `VITE_`**, visible dans le navigateur. L'équipe prépare les changements SQL dans `supabase/migrations/` ; le responsable les applique. Une migration est un fichier qui décrit un changement de structure de la base.

Vérifications avant de partager une modification

Depuis la racine, après avoir installé les dépendances du site et de l'API :

```
npm run typecheck
npm run build
npm run typecheck:api
npm run test:api
npm run build:api
```

`typecheck` détecte les incohérences de types dans le code ; `build` prépare la version à déployer ; `test:api` exécute le test automatique du serveur. **Au 5 septembre 2026 : vérifications TypeScript et compilations du site et de l'API réussies ; 1 test API réussi.** Ces contrôles ne prouvent pas le bon fonctionnement des comptes ou des demandes en production.

Pour valider les vrais parcours, utiliser l'environnement et deux comptes d'essai indiqués par le responsable du projet :

- S'inscrire, confirmer son e-mail, se connecter puis se déconnecter.
- Modifier son profil et sa photo, actualiser la page et vérifier que les changements sont conservés.
- Retrouver le profil réel dans l'annuaire en étant connecté.
- Envoyer une demande au second compte et faire confirmer son enregistrement dans `connection_requests` par le responsable ; la boîte de réception et les notifications restent à développer.
- Vérifier que le second compte ne peut pas modifier le premier profil ni lire son e-mail lorsque celui-ci est masqué.
- Vérifier le site sur mobile et actualiser directement une adresse interne après déploiement.

Transmettre une modification et la publier

L'équipe remet au responsable le code relu, les vérifications réalisées et les éventuels besoins Supabase/Render : migration SQL, nouvelle variable, redirection ou traitement hebdomadaire. **Le responsable prend en charge leur application et la mise en production.**

La configuration prévoit un déploiement automatique après un envoi de code sur la branche GitHub suivie par Render. Toute intégration sur cette branche doit donc être coordonnée avec le responsable. Les développeurs travaillent sur leurs branches et ouvrent une pull request ; ils n'ont pas à reconfigurer les services ni à relancer la migration initiale.

Travailler à plusieurs

Faire chaque modification sur une branche Git dédiée, puis ouvrir une **pull request** : une demande de relecture avant d'intégrer le code à la branche commune. Indiquer le changement, les vérifications effectuées et les limites connues. Mettre à jour ce guide lorsque l'état des fonctionnalités évolue.